Meridian

Real-Time Collaborative Document Editor
Software Requirements Specification (SRS)

1. Project Overview

1.1 Project Name

Meridian

1.2 Project Type

Real-time collaborative document editing web application.

1.3 Purpose

Meridian is a web-based collaborative document editor that allows multiple users to create, edit, and share documents simultaneously.

The system will provide functionality similar to a simplified Google Docs, with a particular focus on solving real-time collaboration problems such as concurrent editing, synchronization, conflict resolution, user presence, document versioning, and offline synchronization.

The project will demonstrate practical knowledge of:

- Real-time distributed systems
- WebSockets
- Operational Transformation (OT) or Conflict-Free Replicated Data Types (CRDTs)
- Rich-text editing
- Distributed state synchronization
- PostgreSQL database design
- Redis pub/sub and presence
- Authentication and authorization
- Offline-first application design
- Version history

2. Goals and Objectives

2.1 Primary Goals

The system shall:

1. Allow users to create and manage documents.

2. Allow multiple users to edit the same document simultaneously.

3. Synchronize document changes in real time.

4. Resolve concurrent editing conflicts automatically.

5. Display the presence of other users.

6. Display other users' cursors/selections.

7. Store documents and their revision history.

8. Support document sharing and permissions.

9. Support comments and discussion threads.

10. Support offline editing and synchronization after reconnection.

2.2 Secondary Goals

The system should:

- Provide a responsive and intuitive editor.
- Minimize synchronization latency.
- Prevent unauthorized document access.
- Recover gracefully from network failures.
- Maintain document consistency across connected clients.
- Provide a scalable architecture suitable for future expansion.

3. Scope

3.1 In Scope

User Management

- User registration
- User login/logout
- Session management
- User profile
- Password management

Document Management

- Create documents
- Rename documents

- Delete documents
- Duplicate documents
- Open documents
- List user's documents
- Search documents
- Document sharing

Collaborative Editing

- Real-time text editing
- Concurrent editing
- Conflict resolution
- Automatic synchronization
- Cursor synchronization
- Selection synchronization
- User presence

Rich Text

- Bold
- Italic
- Underline
- Headings
- Lists
- Links
- Text alignment
- Code formatting
- Undo/redo

Collaboration

- Comments
- Comment threads
- Mention users
- Resolve comments

Versioning

- Document revisions
- Revision timestamps
- Revision authors
- Restore previous version

Offline Support

- Local document cache
- Offline editing
- Change queue
- Synchronization after reconnection
- Conflict resolution

Infrastructure

- WebSocket server
- Redis
- PostgreSQL
- REST/HTTP API
- Real-time synchronization layer

4. Out of Scope

The first version will not include:

- Video conferencing
- Voice calls
- Spreadsheet functionality
- Presentation functionality
- AI-generated documents
- Advanced enterprise administration
- Real-time collaborative drawing
- Native mobile applications
- Complex document formatting equivalent to Microsoft Word
- Large-scale file storage

These may be considered future extensions.

5. User Roles

5.1 Guest

A guest user can:

- Access public landing pages
- Register
- Log in

Guests cannot edit private documents.

5.2 Document Owner

The owner can:

- Create documents
- Edit documents
- Delete documents
- Share documents
- Change collaborator permissions
- Remove collaborators
- View version history

- Restore previous versions
- Manage comments

5.3 Editor

An editor can:

- View shared documents
- Edit document content
- Add comments
- Reply to comments
- View version history

An editor cannot delete the document or modify document ownership.

5.4 Commenter

A commenter can:

- View documents
- Add comments
- Reply to comments
- Resolve their own comments where permitted

A commenter cannot modify document content.

5.5 Viewer

A viewer can:

- View documents
- View comments
- View document history if permitted

A viewer cannot modify document content.

6. Functional Requirements

FR-001 User Registration

The system shall allow a new user to create an account.

Required information:

- Name
- Email address
- Password

The system shall validate:

- Email format
- Password requirements
- Duplicate email addresses

Passwords shall never be stored in plaintext.

FR-002 User Authentication

The system shall allow registered users to log in and log out.

The authentication system shall maintain an authenticated session.

Unauthorized users shall not be able to access private documents.

FR-003 Document Creation

An authenticated user shall be able to create a new document.

A newly created document shall contain:

- Unique document ID
- Document title
- Document content
- Owner ID
- Creation timestamp
- Last modified timestamp

FR-004 Document Editing

Users with editing permission shall be able to modify document content through a rich-text editor.

The editor shall support at minimum:

- Plain text
- Bold
- Italic
- Underline
- Headings
- Ordered lists
- Unordered lists
- Links

7. Real-Time Collaboration Requirements

FR-005 Real-Time Connection

When a user opens a collaborative document, the client shall establish a WebSocket connection with the server.

The WebSocket connection shall be associated with:

- User ID
- Document ID
- Session ID

FR-006 Real-Time Changes

When a user modifies a document, the client shall generate an editing operation/change.

The operation shall be transmitted to the collaboration server.

The server shall:

1. Validate the operation.

2. Determine the document revision.

3. Transform or merge the operation where required.

4. Apply the operation to the authoritative document state.

5. Broadcast the resulting change to other connected clients.

6. Persist the appropriate state/revision.

8. Concurrent Editing

FR-007 Conflict-Free Editing

The system shall support concurrent editing by multiple users.

For example:

User A: "Hello world"

User B simultaneously changes: "Hello world!"

The system shall ensure that both users eventually converge on the same document state.

The implementation shall use one of:

- Operational Transformation (OT), or
- Conflict-Free Replicated Data Type (CRDT).

Recommended implementation

For Meridian, a CRDT-based architecture is recommended because it can simplify offline editing and synchronization.

Possible implementation technologies include:

- Yjs
- Automerge
- Custom CRDT implementation

If the goal of the project is to demonstrate the underlying algorithm, a custom simplified CRDT implementation may be used instead.

9. Document Synchronization

FR-008 Initial Synchronization

When a user opens a document, the client shall obtain the latest document state.

The server shall provide:

- Current document state
- Current revision/version
- Active collaborators
- Relevant comments

FR-009 Incremental Synchronization

After initial synchronization, the client should receive changes incrementally rather than repeatedly downloading the entire document.

The system shall maintain a logical document version/revision.

10. User Presence

FR-010 Presence

The system shall show users currently viewing or editing a document.

Presence information may include:

- User name
- Avatar
- Online/offline status
- Current activity

Example: Alice — editing, Bob — viewing, Charlie — offline

Redis may be used to maintain temporary presence information.

11. Collaborative Cursors

FR-011 Cursor Synchronization

The system shall synchronize the cursor position of connected users.

Users shall be able to see:

- Other users' cursors
- Other users' selections
- User identification/color

Cursor information shall not be persisted as document content.

12. Document Sharing

FR-012 Share Document

The document owner shall be able to share a document with another registered user.

The owner shall specify a permission level:

- Viewer
- Commenter
- Editor

The system shall prevent users from performing actions beyond their permission level.

13. Comments

FR-013 Add Comment

Users with appropriate permission shall be able to add comments to document content.

A comment shall contain:

- Comment ID
- Author ID
- Document ID
- Selected text/range reference
- Comment content
- Creation timestamp
- Resolution status

FR-014 Comment Thread

Users shall be able to reply to comments.

A comment thread shall maintain the relationship between:

- Original comment
- Replies
- Authors
- Timestamps

FR-015 Resolve Comment

Authorized users shall be able to mark a comment thread as resolved.

Resolved comments should no longer appear in the active comment list.

14. Version History

FR-016 Document Versions

The system shall maintain document revision information.

A revision shall contain:

- Revision ID
- Document ID
- Author/user
- Timestamp
- Change information
- Document revision number

FR-017 View History

Authorized users shall be able to view previous document revisions.

The interface should display:

- Revision timestamp
- User responsible for the change
- Revision number

FR-018 Restore Version

Authorized users shall be able to restore a previous document version.

Restoring a version shall create a new revision rather than destroying the historical revision.

15. Offline Editing

FR-019 Offline Mode

The application should continue allowing users to edit a document when the network connection is unavailable.

The client shall maintain a local copy of the document.

FR-020 Offline Change Queue

Changes made while offline shall be stored locally.

The queue shall contain enough information to synchronize the changes when connectivity returns.

FR-021 Reconnection

When connectivity is restored:

1. The client shall reconnect to the WebSocket server.

2. The client shall identify its last synchronized state.

3. Local changes shall be transmitted.

4. Server changes shall be received.

5. Conflicts shall be resolved using the selected synchronization algorithm.

6. The client shall converge on the current document state.

16. Autosave

FR-022 Automatic Persistence

The system shall automatically persist document changes.

Users shall not be required to manually click Save.

The system should minimize unnecessary database writes by batching or periodically persisting changes.

17. Search

FR-023 Document Search

Authenticated users shall be able to search documents they have access to.

Search may include:

- Document title
- Document content
- Owner
- Last modified date

18. Notifications

FR-024 Collaboration Notifications

The system may notify users when:

- They are added to a document
- Someone mentions them in a comment
- Someone replies to their comment
- Their permission changes

19. Non-Functional Requirements

NFR-001 Performance

Normal document changes should be propagated to connected clients with low latency.

Target: p95 synchronization latency < 200 ms under normal development/test conditions.

NFR-002 Scalability

The system architecture shall support multiple WebSocket server instances.

Redis shall be capable of coordinating communication between instances.

NFR-003 Availability

The system should continue operating if an individual WebSocket connection fails.

Clients shall automatically attempt reconnection.

NFR-004 Consistency

All clients editing the same document shall eventually converge to the same document state.

The collaboration layer shall prevent permanent divergent document states.

NFR-005 Security

The system shall:

- Hash passwords securely.

- Authenticate WebSocket connections.
- Authorize document operations.
- Validate all client requests.
- Prevent unauthorized document access.
- Protect against common web vulnerabilities.
- Use HTTPS/WSS in production.

NFR-006 Data Integrity

Document changes shall not be silently lost.

The system shall provide mechanisms for:

- Persistent document storage
- Revision tracking
- Recovery after server restart
- Client reconnection

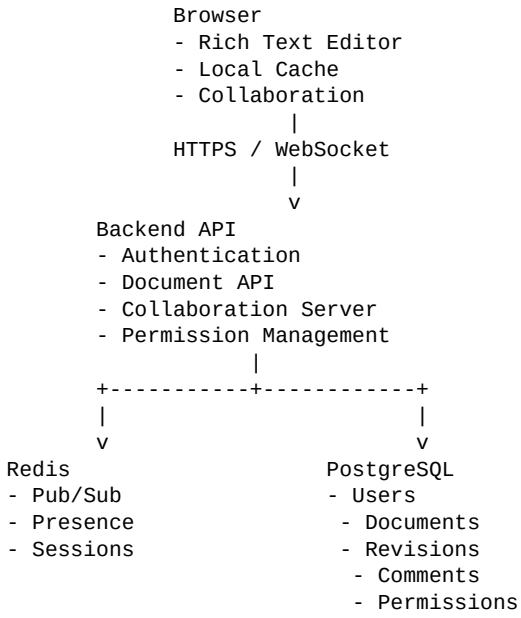
NFR-007 Usability

The editor should provide a simple interface that users can understand without training.

Common operations should require minimal interaction.

20. System Architecture

A suggested architecture:



21. Suggested Technology Stack

Frontend

Recommended:

- React

- TypeScript
- ProseMirror or TipTap
- WebSocket client
- IndexedDB for offline storage

Alternative:

- Vue
- Svelte
- contenteditable

Backend

Recommended:

- Node.js
- TypeScript
- Fastify or Express
- WebSocket library

Alternative:

- Go
- Java/Spring Boot
- Python/FastAPI

Collaboration

Choose one:

Option A — CRDT

- Yjs
- WebSocket provider
- Redis for coordination
- PostgreSQL for persistence

Option B — OT

Implement your own operation transformation layer.

This is more educational but significantly more complex.

22. Database Requirements

users
`id`
`name`
`email`
`password_hash`
`avatar_url`
`created_at`
`updated_at`

documents
id
owner_id
title
content
version
created_at
updated_at

document_members
document_id
user_id
role
created_at
Roles: OWNER, EDITOR, COMMENTER, VIEWER

document_revisions
id
document_id
user_id
version
operation_data
created_at

comments
id
document_id
user_id
parent_comment_id
content
start_position
end_position
resolved
created_at
updated_at

23. API Requirements

Authentication

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET  /api/auth/me
```

Documents

```
GET    /api/documents
POST   /api/documents
GET    /api/documents/:id
PATCH  /api/documents/:id
DELETE /api/documents/:id
```

Sharing

```
GET    /api/documents/:id/members
POST   /api/documents/:id/members
PATCH  /api/documents/:id/members/:userId
DELETE /api/documents/:id/members/:userId
```

Comments

```
GET    /api/documents/:id/comments
POST   /api/documents/:id/comments
PATCH  /api/comments/:id
DELETE /api/comments/:id
```

Versions

```
GET  /api/documents/:id/versions
GET  /api/documents/:id/versions/:version
POST /api/documents/:id/restore/:version
```

24. WebSocket Requirements

Suggested WebSocket endpoint:

```
wss://server.example.com/ws/documents/{documentId}
```

Client to Server Messages

Example document update:

```
{
  "type": "document_update",
  "documentId": "doc_123",
  "clientId": "client_456",
  "version": 42,
  "operation": {}
}
```

Cursor update:

```
{
  "type": "cursor_update",
  "documentId": "doc_123",
  "position": 125,
  "selectionEnd": 140
}
```

Presence:

```
{
  "type": "presence",
  "status": "active"
}
```

Server to Client Messages

Document update:

```
{
  "type": "document_update",
  "version": 43,
  "operation": {}
}
```

Presence:

```
{
  "type": "user_presence",
  "userId": "user_456",
  "status": "online"
}
```

Cursor:

```
{
  "type": "cursor_update",
  "userId": "user_456",
  "position": 125,
  "selectionEnd": 140
}
```

25. Redis Requirements

Redis shall be used for temporary real-time state and communication.

Potential uses:

Pub/Sub

Synchronize events between multiple WebSocket servers (WebSocket Server A -> Redis -> WebSocket Server B).

Presence

Store temporary information such as:

```
presence:{documentId}:{userId}
```

```
{
  "status": "online",
  "lastSeen": "...",
  "cursor": 123
}
```

Presence information does not need to be permanently stored in PostgreSQL.

26. Frontend Requirements

The application shall contain the following major screens: Login, Dashboard, and Editor.

Login screen contains: Meridian title, Email field, Password field, Login button, and a Create account link.

Dashboard screen contains: a header with user profile, a "My Documents" section with a "+ New" button, and a list of documents with titles and last-edited timestamps (e.g. "Project Specification - Edited 2m ago", "Meeting Notes - Edited 1h ago", "Portfolio - Edited yesterday").

Editor screen contains: a header with document title and Share button, a formatting toolbar (Bold, Italic, Underline, Headings, List, Link, Undo, Redo), the document content area showing other users' cursors (e.g. a "Bob" cursor label), and a footer status bar showing the number of active users and connection status (e.g. "3 users editing —

Connected").

27. Collaboration State Model

Each client should maintain information similar to:

- Document State: Content, Version, Pending Operations
- Connection State: Connected, Connecting, Offline
- Presence State: Other Users, Cursors
- Local Persistence: IndexedDB

28. Conflict Resolution Requirements

The system must define how simultaneous operations are handled.

Example:

Initial: "Hello"

User A inserts: "Hello World"

User B inserts: "Hello!"

Both operations may arrive at the server in different orders.

The collaboration algorithm shall ensure that all clients eventually reach an equivalent state.

The algorithm must handle:

- Concurrent insertions
- Concurrent deletions
- Insert/delete conflicts
- Multiple simultaneous users
- Offline changes
- Reconnection
- Duplicate operations
- Out-of-order messages

29. Error Handling

The application shall provide meaningful errors for:

- Invalid login
- Unauthorized document access
- Document not found
- WebSocket disconnection
- Synchronization failure
- Invalid document operation
- Database failure
- Server unavailable

Example message: "Connection lost. Your changes are saved locally. Attempting to reconnect..."

30. Logging and Monitoring

The backend should log:

- Authentication events
- WebSocket connections
- WebSocket disconnections
- Document operations
- Synchronization errors
- Database errors
- Server errors

Sensitive information such as passwords and authentication secrets must not be logged.

31. Testing Requirements

Unit Testing

Test:

- Authentication
- Document operations
- Permission checks
- CRDT/OT operations
- Conflict resolution
- Comment operations

Integration Testing

Test:

- API + PostgreSQL
- WebSocket + Redis
- Authentication + API
- Document persistence

Collaboration Testing

The system should simulate multiple clients (A, B, C) connected to the same server.

Test scenarios:

1. Two users edit simultaneously.

2. Ten users edit simultaneously.

3. A client disconnects during editing.

4. A client reconnects.

5. A client edits while offline.

6. Multiple offline clients reconnect.

7. Users modify the same text range.

8. Users delete overlapping text.

32. Security Requirements

The application shall implement:

Authentication

- Secure password hashing
- Session/token authentication
- Session expiration

Authorization

Every document operation must verify, in order: is the user authenticated, does the user have access, does the user have permission, then perform the operation.

Input Validation

All client-provided data shall be validated on the server.

WebSocket Security

WebSocket connections shall require authentication.

The server shall verify that the authenticated user has access to the requested document.

33. Performance Targets

Metric	Target
API response	< 300 ms p95
Collaboration propagation	< 200 ms p95
Editor startup	< 3 seconds
Reconnection	< 5 seconds
Document initial load	< 2 seconds for normal documents
Concurrent users/document	20+
Concurrent WebSocket clients	1,000+ in a test environment

These are engineering targets rather than guaranteed production limits.

34. MVP Requirements

The first working version should contain only:

Phase 1

- Registration/login
- Document creation
- Document list
- Rich-text editor
- PostgreSQL persistence

Phase 2

- WebSocket connection
- Real-time document synchronization
- Concurrent editing
- CRDT/OT implementation

Phase 3

- Redis
- Presence
- Collaborative cursors
- Reconnection

Phase 4

- Sharing
- Permissions
- Comments
- Version history

Phase 5

- Offline editing
- IndexedDB
- Sync-on-reconnect
- Advanced conflict testing

35. Development Milestones

Milestone 1 — Foundation

Deliver: Repository, Frontend, Backend, Database, Authentication

Milestone 2 — Documents

Deliver: Document CRUD, Editor, Persistence, Basic formatting

Milestone 3 — Real-Time Collaboration

Deliver: WebSockets, Collaboration protocol, CRDT/OT, Concurrent editing

Milestone 4 — Collaboration UX

Deliver: Presence, Cursors, User colors, Connection status

Milestone 5 — Sharing

Deliver: Document invitations, Roles, Permissions

Milestone 6 — Comments and History

Deliver: Comments, Threads, Version history, Restore

Milestone 7 — Offline

Deliver: IndexedDB, Offline editing, Pending operation queue, Reconnection synchronization

Milestone 8 — Production Hardening

Deliver: Tests, Security, Rate limiting, Logging, Monitoring, Docker deployment, CI/CD

36. Acceptance Criteria

The project will be considered successful when:

Authentication

- A user can register and log in.
- Unauthorized users cannot access private documents.

Documents

- A user can create, edit, rename, and delete documents.
- Documents persist after restarting the server.

Collaboration

- Two or more users can edit the same document simultaneously.
- Changes appear on other connected clients without manually refreshing.
- Concurrent changes do not permanently overwrite one another.
- All connected clients eventually converge on the same document state.

Presence

- Users can see who is currently editing.
- Users can see collaborators' cursors/selections.

Sharing

- Owners can share documents.
- Viewer/editor/commenter permissions are enforced.

Versioning

- Previous revisions can be viewed.
- A previous revision can be restored without destroying the revision history.

Offline

- Users can edit while disconnected.
- Changes remain locally stored.
- Changes synchronize after reconnection.

37. Suggested Repository Structure

```
meridian/
■■■ apps/
■   ■■■ web/
■   ■   ■■■ components/
■   ■   ■■■ editor/
■   ■   ■■■ pages/
■   ■   ■■■ hooks/
■   ■   ■■■ services/
■   ■
■   ■■■ server/
■       ■■■ api/
■       ■■■ auth/
■       ■■■ websocket/
■       ■■■ collaboration/
■       ■■■ documents/
■       ■■■ comments/
■
■■■ packages/
■   ■■■ collaboration/
■   ■■■ shared-types/
■   ■■■ validation/
■
■■■ database/
■   ■■■ migrations/
■   ■■■ seeds/
■
■■■ tests/
■   ■■■ unit/
■   ■■■ integration/
■   ■■■ collaboration/
■
■■■ docker/
■
■■■ docker-compose.yml
■■■ README.md
■■■ package.json
```

38. Recommended Final Technology Architecture

For a portfolio-oriented implementation, a practical stack would be:

Frontend: React + TypeScript, with TipTap / ProseMirror, Yjs / CRDT, IndexedDB, and WebSocket.

Backend: Node.js + TypeScript, with a REST API, WebSocket Server, Authentication, and a Collaboration Layer backed by Redis and PostgreSQL.

Infrastructure: Docker, PostgreSQL, Redis, CI/CD, Cloud deployment.

The most technically important component should be the collaboration layer. The portfolio value of Meridian comes less from building another CRUD editor and more from demonstrating that you understand distributed state, concurrent operations, synchronization, persistence, failure recovery, and real-time communication.